

CCDSALG Term 3, AY 2025 – 2026
MCO1 DOCUMENTATION – Convex Hull (Application of Stack Data Structure & Sorting Algorithms)

I. DECLARATION OF INTELLECTUAL HONESTY & ETHICAL USE OF GENERATIVE AI

1. We declare that the project we are submitting is the product of our own work augmented with responsible and ethical use of generative AI as encoded in Table 1 and Table 2 below.
2. We declare that all AI-assisted contributions have been critically examined by our group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full responsibility and accountability for all contents in all our project deliverables.
3. We also declare that each member COOPERATED and contributed EQUALLY to the project as detailed in Table 3 below.
4. No part of our work was shared with another person outside of our group.

// Instruction #1: Encode your names, and affix your e-signature to attest to the declaration above.

Jugno, Daniel Christopher S04 SIGNATURE



Lim, Sean Gabriel S09 SIGNATURE



Chua, Hannelle Celine S09 SIGNATURE



// Instruction #2: Fill-up the column 2 of Table 1 below.

For each of the following tasks, identify the degree of AI use among the following:

- **NONE** - No AI was used for this task
- **SCAFFOLDING** - AI was used to build the foundation for this task, e.g., brainstorming, solution ideation
- **REFINEMENT** - AI was used to review, provide feedback, and revise outputs for this task
- **GENERATION** - AI was used to generate portions of the output for this task

Table 1. Responsible and Ethical Use of AI Details

TASKS	DEGREE OF AI USE & BRIEF EXPLANATION
T1: Understand the project specifications (problem) & assign responsibilities.	NONE: Our group did not use any AI to understand the project specs, role assignment was also done through group discussion
T2: Design, implement and test your stack data structure module.	NONE: The coding, documentation, and testing of the stack data structure was done by members of the group.
T3: Implement and test your TWO sorting algorithms module.	NONE: The coding, documentation, and testing of the sorting algorithms was done by members of the group.
T4: Implement two versions of Graham's Scan algorithm.	NONE: The coding, documentation, and testing of both versions of Graham's scan algorithm was done by members of the group.

T5: Implement two driver programs to test the two Graham's Scan algorithm versions.	<i>NONE: The coding, documentation, and testing of the driver programs was done by members of the group.</i>
T6: Black box test your Graham's Scan Algorithm implementation.	<i>NONE: The blackbox testing and the associated codes for it were made and done by the members of this group without the help of AI.</i>
Task 7. Document your project.	<i>NONE: The documentation process was done by all the members of this group, no additional assistance from generative AI was utilized.</i>

// Instruction #3: Fill up Table 2. List explicitly ALL AI tools that you employed, and justify their purpose/role in your project.

Table 2. List of AI Tools Used

AI TOOL USED (embed the link)	JUSTIFICATION/PURPOSE/ROLE
Tool #1 with embedded link	We used this to

// ... add more lines as you deem necessary

// Instruction #4: Fill up columns 1 and 2 of Table 3.

Table 3. Individual Contributions to the Project

NAME	CONTRIBUTION DETAILS
Jugno, Daniel Christopher	T1: Contributed in discussion of the spec and assignment of roles T2: Created the files for this task T3: Implemented the slow sorting algorithm (selection sort) T4: Implemented the slow graham scan T5: Tested the slow driver program T6: Analyzed the slow algorithm's performance T7: Documented the coding process
Lim, Sean Gabriel	T1: Contributed in discussion of the spec and assignment of roles T2: Did documentation and proofreading T3: Did documentation and proofreading T4: Documentation of the graham_fast and graham_slow T5: Created the initial templates for the test files T6: Generated the test values

	T7: Setup the github and other documentation related procedures
Chua, Hannelle Celine	T1: Contributed in discussion of the spec and assignment of roles T2: Contributed to the implementation of the stack-related files T3: Implemented the fast sorting algorithm (quick sort) T4: Implemented the fast graham scan T5: Tested the fast driver program T6: Analyzed the fast algorithm's performance T7: Documented the coding process

II. SUBMISSION CHECKLIST

// Instruction #5: Fill up Column 3 of Table 4 below.

Put a check mark as specified in the 3rd column of the table below. Please make sure that you use the same file names and that you encoded the appropriate file contents. You may create/add other source files (for example, for reading and writing data).

Table 4. File Submission Checklist

FILE	DESCRIPTION	Put a check mark ✓ below to indicate that you submitted a required file
stack.h	stack data structure header file	✓
stack.c	stack data structure C source file	✓
test_stack.c	driver program for the stack module	✓
sort.h	"slow" and "fast" sorting algo header file	✓
sort.c	"slow" and "fast" sorting algo C source file	✓
test_sort.c	driver program for the sorting module	✓
graham_slow.c	Graham's Scan algorithm slow version (using the "slow" sorting algorithm)	✓
graham_fast.c	Graham's Scan algorithm fast version (using the "fast" sorting algorithm)	✓
test_slow.c	drive program for the "slow" version	✓
test_fast.c	drive program for the for the "fast" version	✓
INPUT1.TXT to INPUT10.TXT	10 sample input files (with increasing values of n)	✓
OUTPUT1.TXT to OUTPUT10.TXT	10 sample corresponding output files	✓
GROUPNUMBER.PDF	the PDF file of this document	✓

III. INSTRUCTIONS ON HOW TO COMPILE & RUN THE PROGRAMS

// Instruction #6: Indicate how to compile your source files, and how to RUN your exe files from the COMMAND LINE.

Examples are shown below highlighted in yellow. Replace them accordingly. Make sure that all your group members test what you typed below because I will follow them verbatim (copy/paste as is). I will initially test your solution using a sample input text file that you submitted. Thereafter, I will run it again using my own test data:

- How to compile from the command line
C:\MCO> gcc -Wall test_slow.c -o slow.exe

```
C:\MCO> gcc -Wall test_fast.c -o fast.exe
```

- How to run from command line

```
C:\MCO>slow
```

```
C:\MCO>fast
```

- Note: test_slow.c and test_fast.c are only made to create one file at a time, if multiple files will be tested, it is suggested to manually change the filename creation located in line 64 of test_fast.c and line 65 of test_slow.c

Next, answer the following questions:

- a. Is there a compilation (syntax error) in your codes? (YES or NO). NO
WARNING: the project will automatically be graded with a score of **0** if there is syntax error in any of the submitted source code files. Please make sure that your submission does not have a syntax error.
- b. Is there any compilation warning in your codes? (YES or NO) NO
WARNING: there will be a 1 point deduction for every unique compiler warning. Please make sure that your submission does not have a compiler warning.

IV. DISCLOSURE OF ERROR(S)

// Instruction #7: Disclose IN DETAIL what is/are NOT working correctly in your solution.

Please be honest about this. NON-DISCLOSURE will result in severe point deduction! Explain briefly the reason why your group was not able to make it work.

The following are NOT working (buggy):

a.

b.

We were not able to make them work because:

a.

b.

V. TEST RESULTS

// Instruction #8: Fill up Table 5 below.

Based on the exhaustive empirical testing that you did, fill-up the Comparison Table below that shows the performance between the “slow” version versus the “fast” version. Test for 10 different values of n , starting with $n = 2^6 = 64$ points.

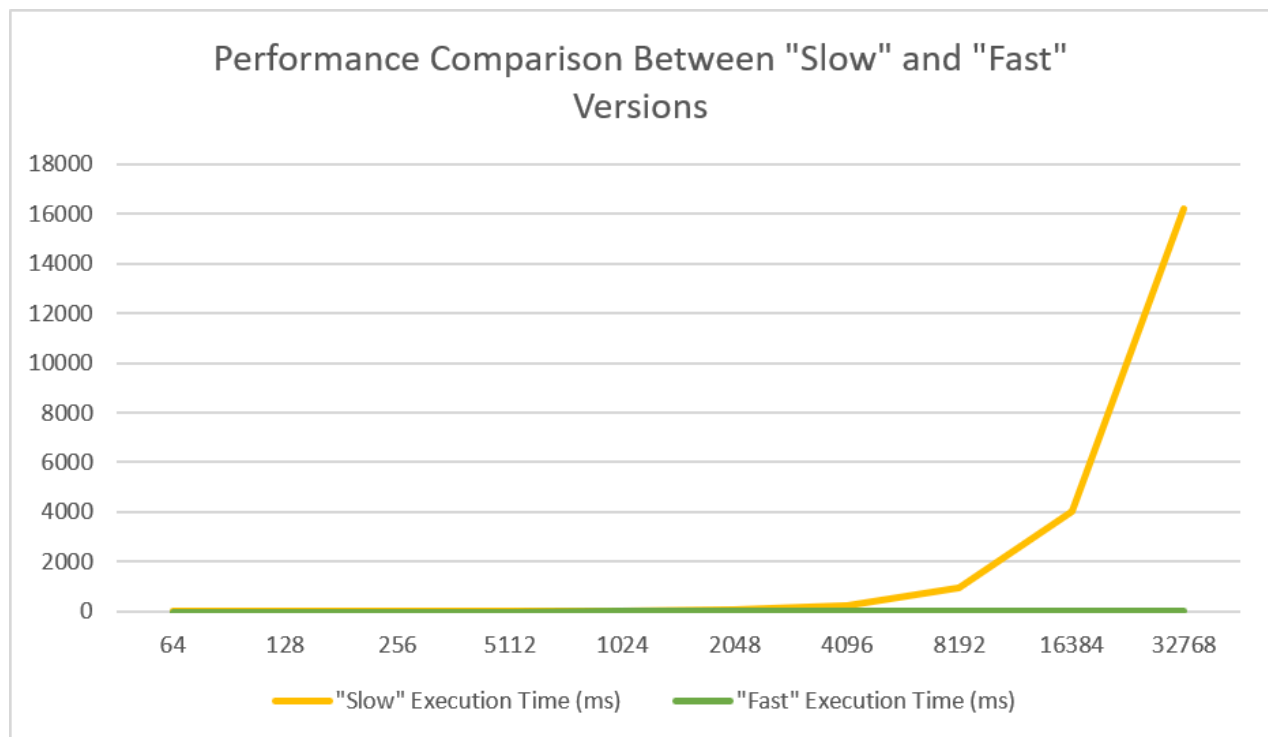
Table 5. Performance Comparison Between “Slow” and “Fast” Versions

Test Case #	n (input size)	"Slow" version execution time in ms	"Fast" version execution time in ms
1	$2^6 = 64$	0.000000	0.000000
2	2^7	0.000000	0.000000
3	2^8	1.000000	0.000000
4	2^9	4.000000	0.000000
5	2^{10}	15.000000	1.000000
6	2^{11}	58.000000	2.000000
7	2^{12}	241.000000	2.000000
8	2^{13}	947.000000	4.000000
9	2^{14}	4028.000000	10.000000
10	$2^{15} = 32768$	16191.000000	22.000000

NOTE: Make sure that you fill-up the table properly. It contributes 4 out of 15 points for the Documentation.

// Instruction #9: Provide a graph showing a visual comparison of the "slow" and "fast" versions.

Create a graph (for example using Excel) based on the Comparison Table that you filled-up above. The x-axis should be the values of n and the y axis should be the execution time in milliseconds (ms). **There should be two line graphs superimposed on the same xy axes, one for the "slow" and the other for the "fast".** DO NOT SUBMIT TWO separate graphs. Replace the image below with your own.



NOTE: Make sure that you provide a graph based on your comparison table data above. It contributes 4 out of 15 points for the Documentation.

VI. ANALYSIS OF RESULTS

// Instruction #10: Compare and analyze the growth rate behaviors of the "slow" and "fast" versions based on the Comparison Table and the graphs above.

Answer the following questions:

a. What do you think is the growth rate behavior of the “slow” version?

The growth rate for slow version is likely to be n^2 as the time it takes increases by a significant amount with increasing n

b. What do you think is the growth rate behavior of the “fast” version?

The growth rate for the fast version is likely to be $n \log n$ as the time it takes to complete the convex hull increases relatively slowly when compared to the slow version.

c. What do you think is/are the factor/s that make the “fast” version compute the results faster than the “slow” version?

The slow version uses selection sort which has needs to go through more comparisons compared to the quick sort implemented in the fast version

NOTE: Make sure that you provide cohesive answers to the three questions above. This part contributes 4 out of 15 points for the Documentation.

VII. SELF-EVALUATION

// Instruction #11: Fill-up Table 6 below.

Refer to the rubric in the project specs. It is suggested that you do an individual self-assessment first. Thereafter, compute the average evaluation for your group, and encode it below.

Table 6. Self-Evaluation

REQUIREMENT	AVE. OF SELF-ASSESSMENT
1. Stack	<u>20</u> (max. 20 points)
2. Sorting algorithms	<u>10</u> (max. 10 points)
3. Graham’s Scan algorithm	<u>40</u> (max. 40 points)
4. Project Documentation and Analysis	<u>15</u> (max. 15 points)
5. Compliance with Instructions	<u>5</u> (max. 5 points)

TOTAL SCORE _____ over 90.

Note that the remaining 10 points (over 100) will come from the project demo.

NOTE: The evaluation that the instructor will give is not necessarily going to be the same as what you indicated above. The self-assessment is for your own reference only...

– The End –

サルバド
ル・フロランテ